API SECURITY RISK ANALYSIS REPORT

Tested API:

JSON Placeholder

Prepared by:

Narendra Varma Palepu

Tools Used:

- Postman

- Browser DevTools

Objective:

To analyse public API endpoints and identify potential security risks such as data exposure, missing authentication, and improper access control.

Methodology

- The following approach was used to perform API security testing:

1. Selected a public test API (JSON Placeholder)
2. Tested endpoints using browser and Postman
3. Observed API responses and data structure
4. Checked for authentication and authorization controls
5. Identified potential security risks
6. Documented findings with evidence (screenshots)

Endpoints Tested:

- /users
- /posts
- /comments

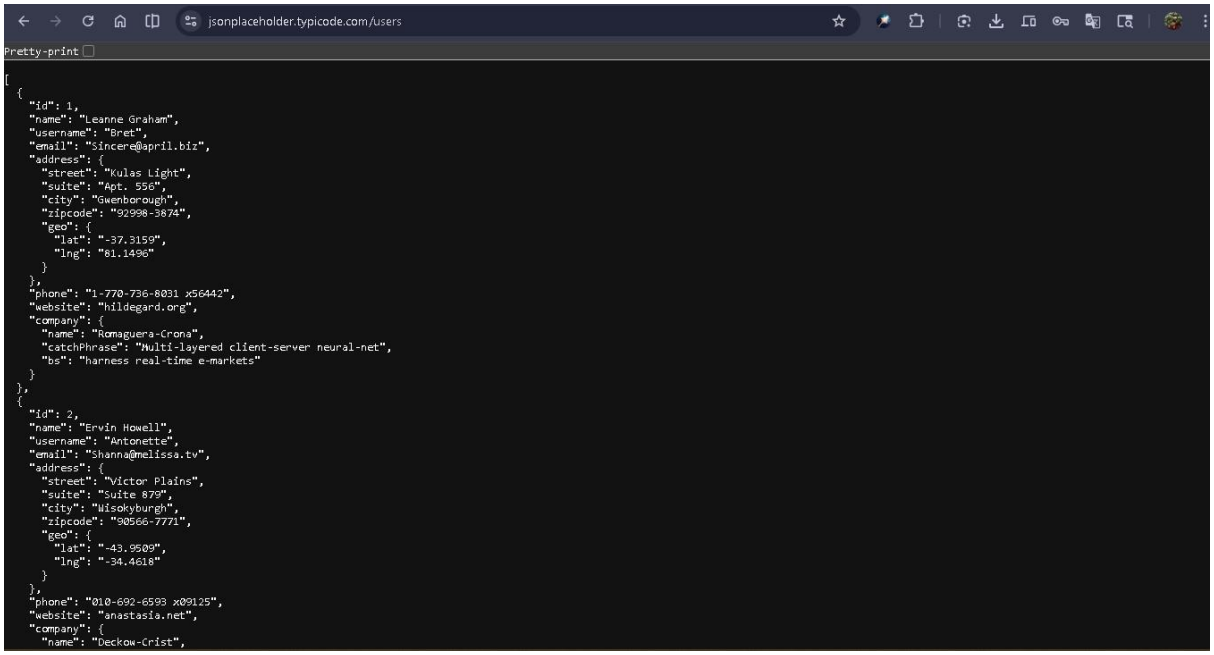


Figure (1): API Users Endpoint Response using Browser

Observation:

- The API endpoint /users was accessed through the browser, and it returned a list of user records in JSON format. The response included sensitive details such as names, email addresses, and location data. No authentication or authorization mechanism was required to access this data, indicating an open API endpoint.

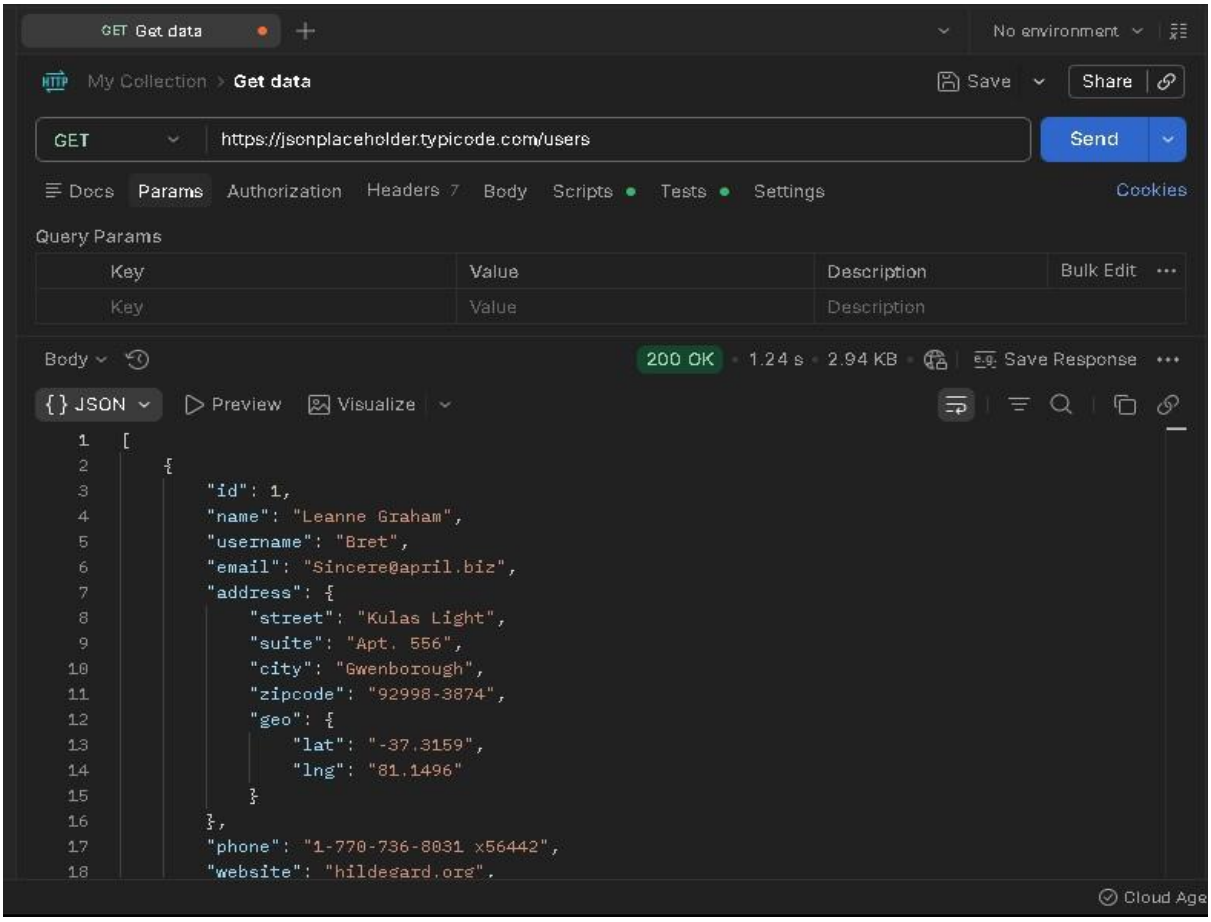
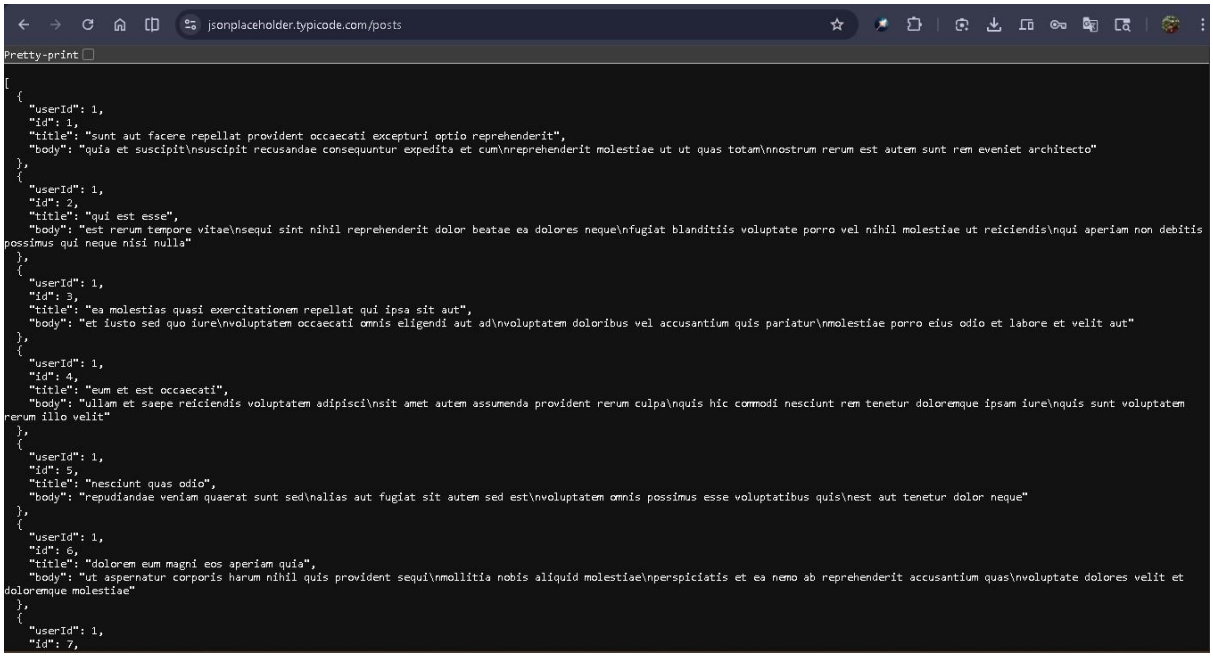


Figure (2): API Users Endpoint Response using Postma

Observation:

- The same /users endpoint was tested using Postman, confirming that the API allows direct requests without any authentication. The response status was 200 OK, and full user data was retrieved successfully. This verifies that the API is publicly accessible and does not enforce access control.




```
[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
  },
  {
    "userId": 1,
    "id": 2,
    "title": "qui est esse",
    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis
possimus qui neque nisi nulla"
  },
  {
    "userId": 1,
    "id": 3,
    "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
    "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et labore et velit aut"
  },
  {
    "userId": 1,
    "id": 4,
    "title": "eum et est occaecati",
    "body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic commodi nesciunt rem tenetur doloremque ipsam iure\nquis sunt voluptatem
rerum illo velit"
  },
  {
    "userId": 1,
    "id": 5,
    "title": "nesciunt quas odio",
    "body": "repudiandae veniam quaerat sunt sed\nalias aut fugiat sit autem sed est\nvoluptatem omnis possimus esse voluptatibus quis\nest aut tenetur dolor neque"
  },
  {
    "userId": 1,
    "id": 6,
    "title": "dolorem eum magni eos aperiam quia",
    "body": "ut aspernatur corporis harum nihil quis provident sequi\nmollitia nobis aliquid molestiae\nperspiciatis et ea nemo ab reprehenderit accusantium quas\nvoluptate dolores velit et
doloremque molestiae"
  },
  {
    "userId": 1,
    "id": 7,
```


Figure (3): API Posts Endpoint Response using Browser

Observation:

- The /posts endpoint returned a large dataset of posts created by users. Each post included user Id, title, and body content. The data was accessible without authentication, and multiple records were returned in a single response, indicating bulk data exposure.

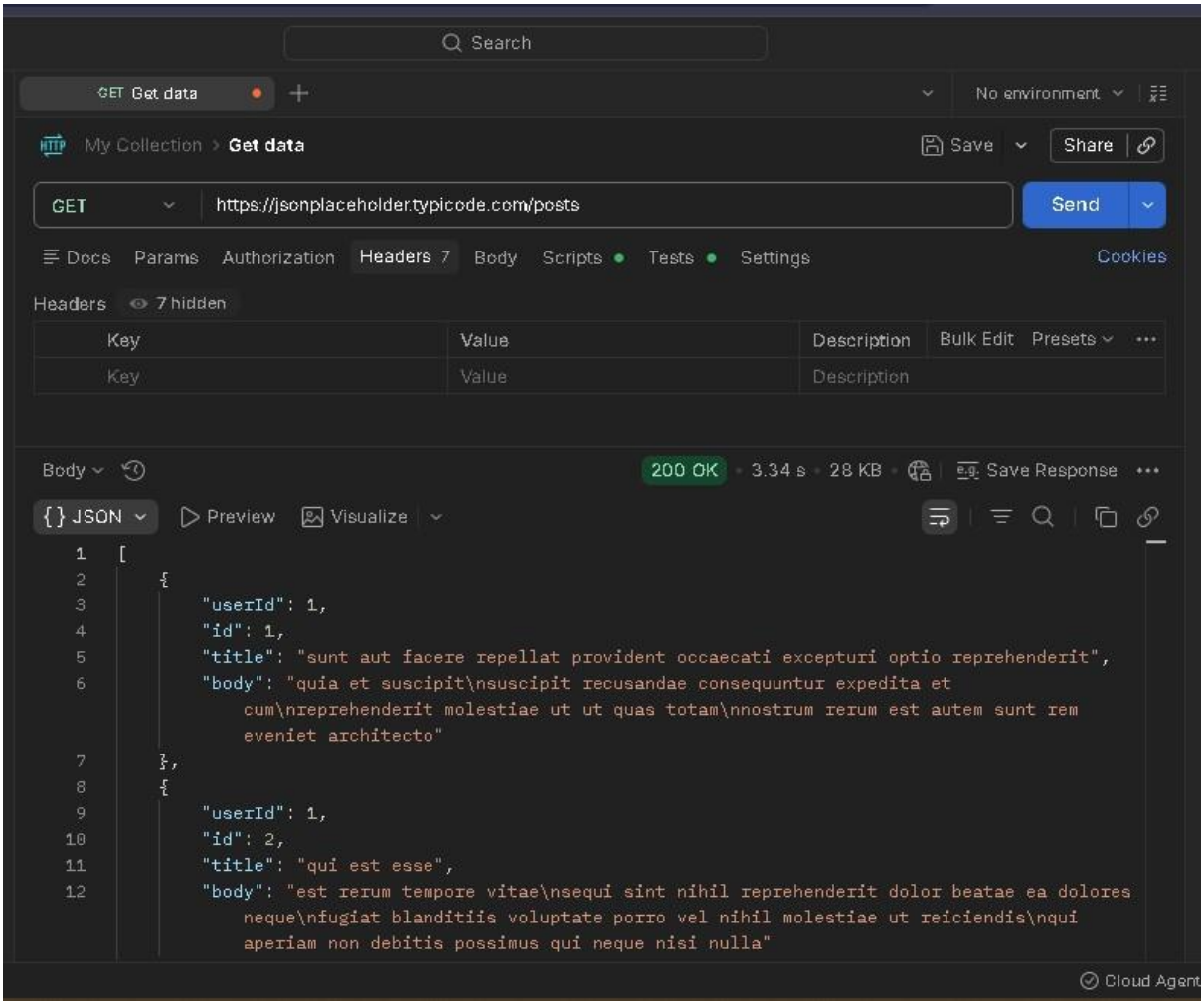
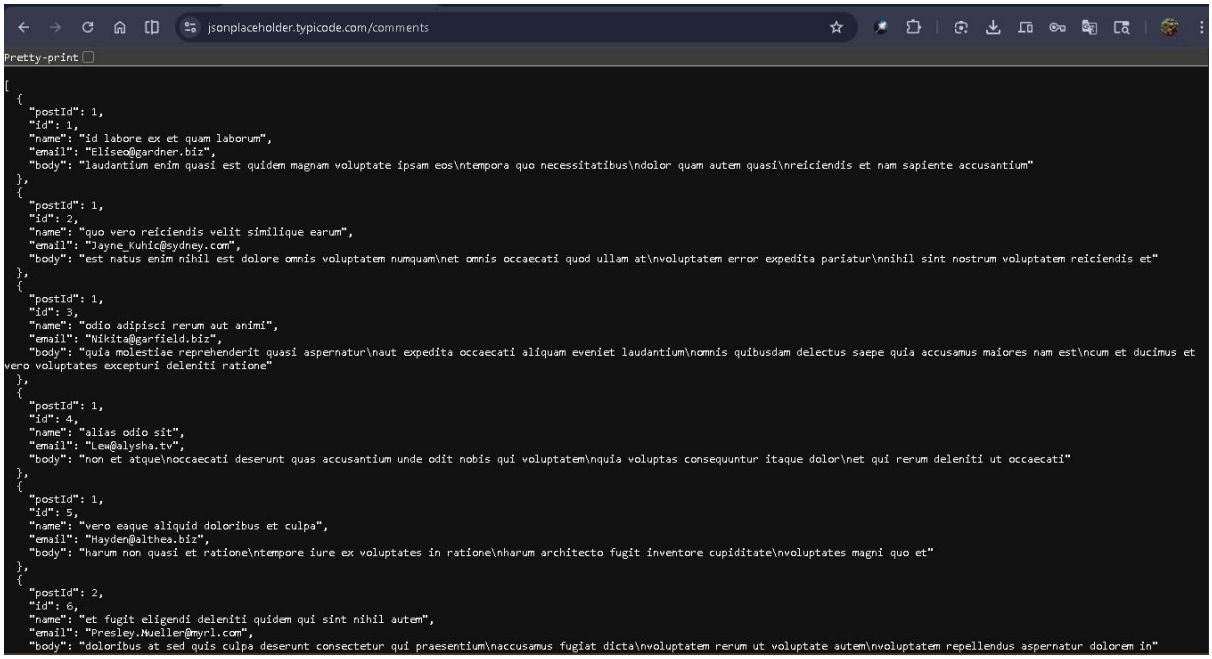


Figure (4): API Posts Endpoint Response using Postman

Observation:

- Testing the /posts endpoint using Postman showed that the API responds with a full dataset of posts without any access restrictions. This confirms the absence of authentication and demonstrates the risk of automated data scraping.




```
[
  {
    "postId": 1,
    "id": 1,
    "name": "id labore ex et quam laborum",
    "email": "Eliseo@gardner.biz",
    "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
  },
  {
    "postId": 1,
    "id": 2,
    "name": "quo vero reiciendis velit similique earum",
    "email": "Jayne_Kuhic@sydney.com",
    "body": "est natus enim nihil est dolore omnis voluptatem numquam\net omnis occaecati quod ullam at\nvoluptatem error expedita pariatur\nnihil sint nostrum voluptatem reiciendis et"
  },
  {
    "postId": 1,
    "id": 3,
    "name": "odio adipisci rerum aut animi",
    "email": "Nikita@garfield.biz",
    "body": "quia molestiae reprehenderit quasi aspernatur\naut expedita occaecati aliquam eveniet laudantium\nomnis quibusdam delectus saepe quia accusamus maiores nam est\ncum et ducimus et
vero voluptates excepturi deleniti ratione"
  },
  {
    "postId": 1,
    "id": 4,
    "name": "alias odio sit",
    "email": "Lew@alysha.tv",
    "body": "non et atque\noccaecati deserunt quas accusantium unde odit nobis qui voluptatem\nquia voluptas consequuntur itaque dolor\net qui rerum deleniti ut occaecati"
  },
  {
    "postId": 1,
    "id": 5,
    "name": "vero eaque aliquid doloribus et culpa",
    "email": "Hayden@althea.biz",
    "body": "harum non quasi et ratione\ntempore iure ex voluptates in ratione\nharum architecto fugit inventore cupiditate\nvoluptates magni quo et"
  },
  {
    "postId": 2,
    "id": 6,
    "name": "et fugit eligendi deleniti quidem qui sint nihil autem",
    "email": "Presley.Mueller@myrl.com",
    "body": "doloribus at sed quis culpa deserunt consectetur qui praesentium\naccusamus fugiat dicta\nvoluptatem rerum ut voluptate autem\nvoluptatem repellendus aspernatur dolorem in"
```


Figure 5: API Comments Endpoint Response using Browser

Observation:

- The /comments endpoint returned user comments including names, email addresses, and message content. The presence of email addresses in the response indicates exposure of sensitive user information. No authentication was required to access this endpoint.

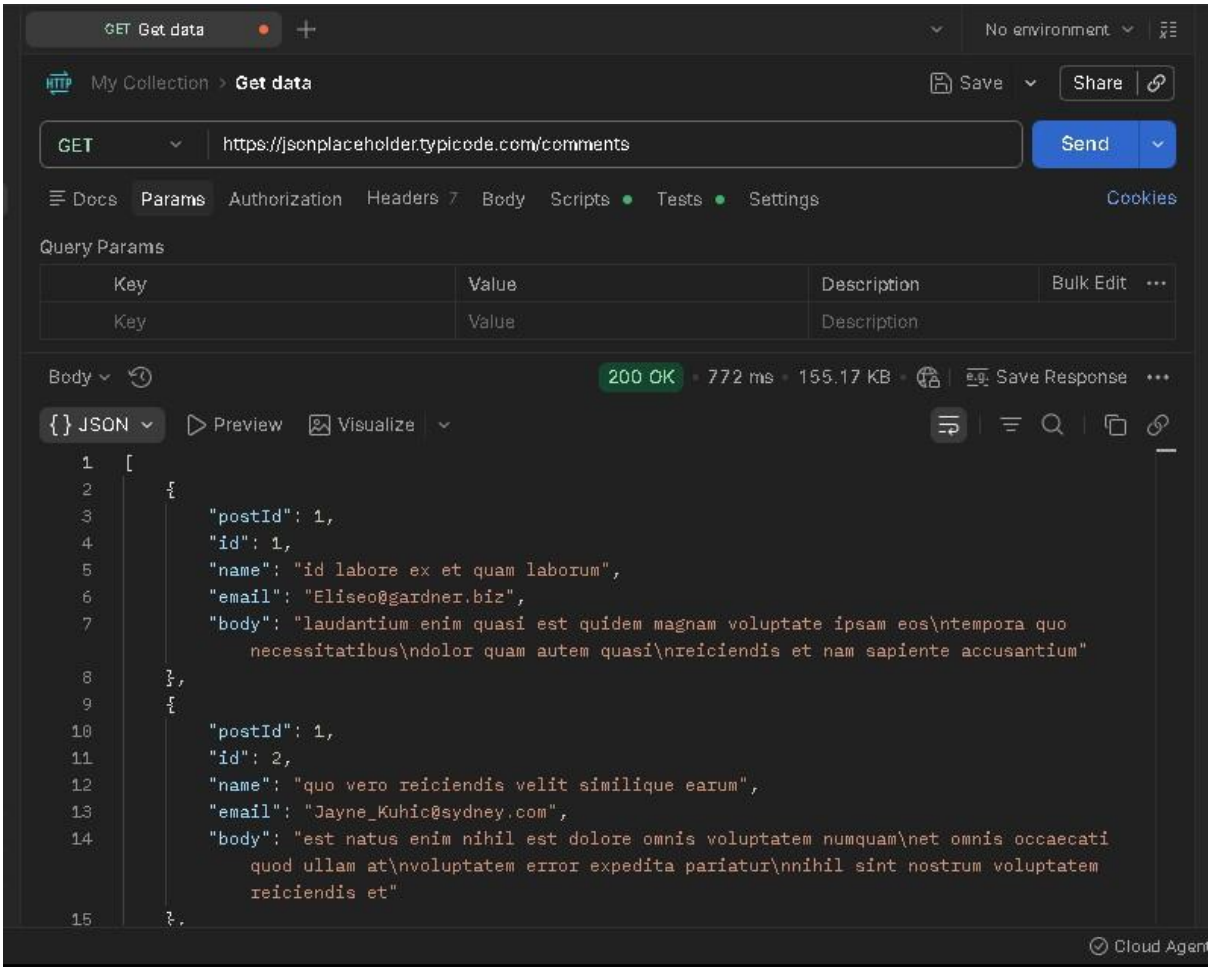


Figure (6): API Comments Endpoint Response using Postman

Observation:

- The /comments endpoint was tested using Postman and returned a successful response (200 OK). The API exposed comment data along with user email addresses without any security controls. This confirms the risk of data leakage and potential misuse for phishing or spam attacks.

Overall Observation

All tested API endpoints were publicly accessible without authentication, exposing structured and potentially sensitive data. The lack of security controls increases the risk of unauthorized access, data scraping, and misuse.

IDENTIFIED RISKS

1. No authentication:

Description:

The API allows access to endpoints without requiring any authentication.

Risk Level: High

Impact:

Unauthorized users can access sensitive data without any restrictions.

Recommendation:

Implement authentication mechanisms such as API keys, tokens, or

OAuth.

2. Excessive Data Exposure:

Description:

The API returns more data than necessary in responses.

Risk Level: Medium

Impact:

Attackers can collect large amounts of information through automated requests.
Recommendation: Limit response data and use filtering or pagination.

3. **Exposure Of Sensitive Data:**

Description:

The API returns more data than necessary in responses.

Risk Level: Medium

Impact:

Attackers can collect large amounts of information through automated requests.

Recommendation:

Limit response data and use filtering or pagination.

4. **Missing Rate Limiting:**

Description:

No restriction on number of API requests.

Risk Level: Medium **Impact:**
Attackers can abuse the API for data scraping or denial-of-service attacks.

Recommendation:

Implement rate limiting to control request frequency.

Business Impact:

The identified vulnerabilities can lead to unauthorized data access, user privacy violations, and potential misuse of sensitive information. Attackers may exploit open APIs to collect large amounts of user data, leading to spam, phishing attacks, and reputational damage to the organization.

Risk Severity Classification

1. Open API Endpoints – HIGH

2. Exposure of User Emails – HIGH

3. Lack of Authentication – HIGH

4. Excessive Data Exposure – MEDIUM

5. No Rate Limiting – MEDIUM

Remediation Suggestions:

• Implement authentication mechanisms such as API keys or tokens

• Restrict access to sensitive endpoints

• Apply rate limiting to prevent abuse

• Limit data exposure by returning only required fields

• Validate all incoming requests

• Monitor API activity for suspicious behaviour

CONCLUSION

- The analysis of the JSONPlaceholder API revealed multiple security weaknesses,
- including lack of authentication, excessive data exposure, and absence of rate limiting.
- Although this API is designed for testing purposes, similar issues in real world APIs
- can lead to serious security breaches.
  - Proper implementation of authentication, access control, and data protection